UNIVERSIDAD TECNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial

INFORME DE EXAMEN PRACTICO INTEGRADOR

Tema:	Sistema de Gestión - Taller Mecánico (Grupo L)
Unidad Curricular:	BASICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Chacha Chango Victor Manuel, Landeta Tapia Edison Paul, Miranda Garces Alan Imanol
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Caiza
Fecha:	Ambato, 15 Abril 2026

1. Propósito del examen

El presente informe documenta el desarrollo del Examen Practico Integrador de C++ correspondiente al Grupo L - Taller Mecánico. El examen integra los contenidos del parcial: análisis de problemas, estructuras de control, validación de datos, funciones, arreglos, recursividad, programación orientada a objetos, encapsulamiento, constructores, depuración y defensa técnica.

El sistema desarrollado permite controlar ordenes de trabajo, repuestos, horas de mano de obra, estados y costos finales de un taller mecánico. Además, incluye un módulo obligatorio de Ahorcado con vocabulario técnico del taller.

2. Competencias evaluadas

- Analizar el problema y transformar el enunciado en entradas, procesos, salidas y reglas de negocio.
- Diseñar clases, objetos, constructores, atributos privados, getters y setters con criterio técnico.
- Construir un menú funcional, robusto y validado para el caso principal y el módulo Ahorcado.
- Aplicar correctamente condicionales, ciclos, arreglos, búsqueda manual y ordenamiento manual.
- Utilizar recursividad en una operación real del sistema.
- Depurar fragmentos defectuosos y justificar técnicamente la corrección.
- Trabajar con metodología de desarrollo y evidenciar versionamiento con GitHub Desktop.

3. Metodología de desarrollo

Fase 0 - Análisis y prueba de escritorio

Se analizo el caso del Grupo L identificando las entidades principales, sus atributos, las reglas de negocio y los casos limite. Se definieron las dos clases requeridas: OrdenTaller y Repuesto, con sus respectivos atributos y tipos de dato en C++.

Reglas de negocio definidas:

- Costo mano de obra = horasManoObra x costoPorHora
- Costo repuestos = suma de (precioUnitario x cantidad) de cada repuesto
- Subtotal = costoManoObra + costoRepuestos
- Descuento: preventivo 10%, express 5%, correctivo sin descuento
- Impuesto = (subtotal - descuento) x 15%
- Valor final = subtotal - descuento + impuesto
- Clasificación: Menor (<$100), Moderada ($100-$500), Mayor (>$500)

Fase 1 - Estructura base del proyecto

Se creo el archivo main.cpp con las dos clases vacías, pero correctamente estructuradas con encapsulamiento completo: atributos privados, constructor, getters y setters. Se implemento el menú principal con todas las opciones y se verifico que compilara sin errores.

Fase 2 - Registros y validaciones

Se implementaron las funciones registrarRepuesto() y registrarOrden() con validaciones completas. Cada entrada del usuario es validada: precios mayores a cero, horas mayores a cero, tipo de servicio dentro de las opciones validas, código de repuesto único. Se implemento también mostrarOrden() con calculo automático de costos.

Fase 3 - Búsqueda, ordenamiento e historial

Se implementaron tres funciones: buscarOrden() permite buscar por placa o cedula del cliente. ordenarOrdenes() implementa el algoritmo de ordenamiento burbuja manual por mayor valor final. verHistorial() muestra todas las órdenes y detecta las de baja rentabilidad (valor final menor a $100).

Fase 4 - Recursividad y fragmento defectuoso

Se implemento la función recursiva sumarTotalOrdenes() que acumula el valor final de todas las ordenes registradas. Se corrigió el fragmento defectuoso totalOrden() que contenia 3 errores lógicos y de sintaxis.

Fase 5 - Modulo Ahorcado

Se implemento el módulo completo de Ahorcado con vocabulario del taller mecánico. Incluye dos niveles de dificultad, validación de entradas, historial de partidas y conteo de victorias.

Fase 6 - README

Se creo el archivo README.md en el repositorio con la información del equipo, caso asignado, metodología seguida, partes implementadas y herramientas utilizadas.

Fase 7 - Prueba de escritorio

Se documento la prueba de escritorio de un caso critico calculado manualmente, verificando que el programa arroja el mismo resultado.

4. Diseño de clases

Clase Repuesto

Atributo	Tipo	Descripcion
codigo	string	Identificador unico del repuesto
nombre	string	Nombre del repuesto
precioUnitario	double	Precio por unidad
cantidad	int	Stock disponible

Clase OrdenTaller

Atributo	Tipo	Descripcion
placa	string	Placa del vehiculo
cliente	string	Nombre del cliente
cedula	string	Cedula del cliente
tipoServicio	string	preventivo/correctivo/express
horasManoObra	double	Horas trabajadas
costoPorHora	double	Tarifa por hora
estado	string	pendiente/en proceso/terminada
repuestosUsados[5]	string[]	Nombres de repuestos usados
cantidadRepuestos[5]	int[]	Cantidades de cada repuesto
numRepuestos	int	Contador de repuestos en la orden

5. Fragmento defectuoso corregido

El fragmento asignado al Grupo L corresponde a la funcion totalOrden(). A continuacion se muestra el codigo original con errores y la version corregida.

Codigo original con errores

```
double totalOrden(double manoObra, double repuestos, double descuento){
    double total = manoObra + repuestos - descuento;
    total = total + total * 0.15    // ERROR 1: falta punto y coma
    if(total < 0) total = 0;
    return manoObra;            // ERROR 2: retorna manoObra en lugar de total
}                               // ERROR 3: no valida que descuento <= subtotal
```

Errores identificados

Error	Tipo	Descripcion
1	Sintaxis	Falta punto y coma al final de la linea del impuesto
2	Logico	Retorna manoObra en lugar de total, resultado siempre incorrecto
3	Logico	No valida que el descuento no supere el subtotal, puede generar valores negativos

Código corregido

```
double totalOrden(double manoObra, double repuestos, double descuento){
    double subtotal = manoObra + repuestos;
    if(descuento > subtotal) descuento = subtotal; // correccion error 3
    double total = subtotal - descuento;
    total = total + total * 0.15;  // correccion error 1: se agrega ;
    if(total < 0) total = 0;
    return total;                  // correccion error 2: retorna total
}
```

6. Función recursiva

Se implemento la funcion sumarTotalOrdenes() que calcula de forma recursiva el valor total facturado sumando el valor final de todas las ordenes registradas en el sistema.

```
double sumarTotalOrdenes(int indice) {
if (indice >= numOrdenes) return 0;   // caso base
return ordenes[indice].getValorFinal(repuestos, numRepuestos)
+ sumarTotalOrdenes(indice + 1); // llamada recursiva
}
```

La función recibe el índice actual, suma el valor final de la orden en esa posición y llama recursivamente con el siguiente índice. Cuando el índice supera el número de ordenes retorna 0 como caso base.

7. Prueba de escritorio

Se documento el cálculo manual de un caso crítico para verificar la corrección de la lógica implementada.

Datos de entrada

Dato	Valor
Tipo de servicio	preventivo
Horas de mano de obra	3
Costo por hora	$25.00
Repuesto	Filtro de aceite x1
Precio repuesto	$15.50

Desarrollo paso a paso

1. Costo de mano de obra: 3 horas x $25.00 = $75.00
2. Costo de repuestos: 1 x $15.50 = $15.50
3. Subtotal: $75.00 + $15.50 = $90.50
4. Descuento (preventivo 10%): $90.50 x 10% = $9.05 -> $90.50 - $9.05 = $81.45
5. Impuesto (15%): $81.45 x 15% = $12.22
6. Valor final: $81.45 + $12.22 = $93.67
7. Clasificación: $93.67 < $100 -> Reparación MENOR

Resultado

Campo	Valor
Costo mano de obra	$75.00
Costo repuestos	$15.50
Descuento	$9.05
Impuesto	$12.22
Valor final	$93.67
Clasificación	MENOR

El programa arrojo exactamente $93.67 y clasificación Menor, confirmando que la lógica implementada es correcta.

8. Modulo Ahorcado

Se implemento el módulo de Ahorcado integrado al sistema del taller mecánico. El vocabulario utilizado pertenece al dominio del taller.

Palabras nivel fácil

ORDEN, PLACA, MOTOR, FRENO, LLANTA

Palabras nivel difícil

CARBURADOR, TRANSMISION, AMORTIGUADOR, DIFERENCIAL, ALTERNADOR

Funcionalidades implementadas

- Menú propio con opciones: jugar, ver historial, ver instrucciones
- Dos niveles de dificultad con conjuntos de palabras distintos
- Validación de entradas: solo letras, sin repetición de letras
- Visualización del ahorcado progresivo con 6 intentos máximos
- Historial de partidas con conteo de victorias del jugador y CPU
- Vocabulario adaptado al caso del Grupo L (taller mecánico)

9. Algoritmo, Pseudocódigo y Diagrama de Flujo

Algoritmo (PSeInt)

Inicio

Definir e inicializar arreglos para un máximo de 50 repuestos (código, nombre, precio, cantidad en stock).

Definir e inicializar arreglos para un máximo de 50 órdenes de trabajo (placa, cliente, cédula, tipo de servicio, horas, costo por hora, repuestos utilizados).

Definir e inicializar arreglos para el juego del Ahorcado (banco de palabras fáciles, palabras difíciles y registros de historial).

Iniciar un ciclo Repetir para el Menú Principal.

Mostrar en pantalla las opciones del menú (1. Registrar repuesto, 2. Registrar orden, 3. Buscar orden, 4. Ordenar órdenes, 5. Ver historial, 6. Total recursivo, 7. Ahorcado, 0. Salir).

Leer la opción ingresada por el usuario.

Evaluar la opción seleccionada (Según Hacer):

Si es 1: Validar que haya espacio. Pedir el código del repuesto (validando que no exista previamente), nombre, precio (mayor a cero) y cantidad (mayor o igual a cero). Guardar los datos.

Si es 2: Validar que haya espacio. Pedir los datos de la orden (placa, cliente, cédula, tipo de servicio, horas trabajadas y costo por hora). Iniciar un ciclo secundario para preguntar si desea agregar repuestos. Si la respuesta es 's', mostrar repuestos disponibles, leer el repuesto a usar y la cantidad, y guardarlos dentro de la orden.

Si es 3: Preguntar si buscará por placa (1) o cédula (2). Leer el dato a buscar. Recorrer el arreglo de órdenes; si se encuentra coincidencia, llamar al subproceso para calcular el valor final y mostrar todos los detalles de esa orden.

Si es 4: Verificar que existan al menos dos órdenes. Iniciar ciclos anidados (Método de Burbuja), llamar al subproceso de cálculo para comparar el costo de las órdenes contiguas. Si una orden posterior tiene mayor costo, intercambiar todos los datos de sus arreglos respectivos para dejarlas ordenadas de mayor a menor valor.

Si es 5: Verificar si hay órdenes. Recorrer y mostrar el resumen de todas las órdenes registradas llamando al subproceso de valor. Posteriormente, recorrer nuevamente y mostrar solo aquellas órdenes cuya rentabilidad sea baja (valor inferior a $100).

Si es 6: Llamar al subproceso recursivo enviando la posición inicial (1). Este subproceso se llamará a sí mismo sumando el valor de la orden actual con el de las siguientes hasta recorrer todas. Mostrar el total facturado sumado.

Si es 7: Mostrar un submenú para el Ahorcado (1. Jugar, 2. Historial).

Si elige jugar: Pedir la dificultad, seleccionar una palabra aleatoria, iniciar un ciclo de 6 intentos pidiendo letras y llamar al subproceso gráfico para mostrar el muñeco según los errores. Finalizar y guardar resultado.

Si elige historial: Mostrar el registro de las partidas jugadas.

Si es 0: Mostrar mensaje indicando que el sistema se cerrará.

De otro modo: Mostrar mensaje de "Opción inválida".

Repetir el ciclo principal hasta que la opción elegida sea igual a 0.

Fin

Pseudodocódigo

```
// ==================== FUNCIONES DE CALCULO ====================

SubProceso valor_final <- CalcularValorOrden(idx, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

    Definir manoObra, costoRepuestos, subtotal, descuento, impuesto Como Real

    Definir i, j Como Entero

    manoObra <- ord_horas[idx] * ord_costo_hora[idx]

    costoRepuestos <- 0



    Para i <- 1 Hasta ord_num_reps[idx] Hacer

        Para j <- 1 Hasta numRepuestos Hacer
```

```
            Si ord_rep_nombres[idx, i] = rep_nombre[j] Entonces
                costoRepuestos <- costoRepuestos + (rep_precio[j] * ord_rep_cants[idx, i])
            FinSi
        FinPara
    FinPara

    subtotal <- manoObra + costoRepuestos
    descuento <- 0

    Si ord_tipo[idx] = "preventivo" Entonces
        descuento <- subtotal * 0.10
    Sino
        Si ord_tipo[idx] = "express" Entonces
            descuento <- subtotal * 0.05
        FinSi
    FinSi

    impuesto <- (subtotal - descuento) * 0.15
    valor_final <- subtotal - descuento + impuesto
FinSubProceso

SubProceso clasif <- ObtenerClasificacion(valor)
    Definir clasif Como Caracter
    Si valor < 100 Entonces
        clasif <- "Menor"
    Sino
```

```
        Si valor <= 500 Entonces

            clasif <- "Moderada"

        Sino

            clasif <- "Mayor"

        FinSi

    FinSi

FinSubProceso



// ==================== FUNCION RECURSIVA ====================



SubProceso suma <- SumarTotalOrdenes(indice, numOrdenes, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

    Definir valorActual, suma Como Real

    Si indice > numOrdenes Entonces

        suma <- 0

    Sino

        valorActual <- CalcularValorOrden(indice, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

        suma <- valorActual + SumarTotalOrdenes(indice + 1, numOrdenes, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

    FinSi

FinSubProceso



// ==================== MODULO AHORCADO ====================



SubProceso MostrarAhorcadoGrafico(errores)

    Escribir ""
```

```
Segun errores Hacer
    0:
        Escribir "  +---+"
        Escribir "  |   |"
        Escribir "      |"
        Escribir "      |"
        Escribir "      |"
        Escribir "      |"
        Escribir "=========="
    1:
        Escribir "  +---+"
        Escribir "  |   |"
        Escribir "  O   |"
        Escribir "      |"
        Escribir "      |"
        Escribir "      |"
        Escribir "=========="
    2:
        Escribir "  +---+"
        Escribir "  |   |"
        Escribir "  O   |"
        Escribir "  |   |"
        Escribir "      |"
        Escribir "      |"
        Escribir "=========="
    3:
```

```
    Escribir "  +---+"
    Escribir "  |   |"
    Escribir "  O   |"
    Escribir " /|   |"
    Escribir "      |"
    Escribir "      |"
    Escribir "=========="
4:
    Escribir "  +---+"
    Escribir "  |   |"
    Escribir "  O   |"
    Escribir " /|\  |"
    Escribir "      |"
    Escribir "      |"
    Escribir "=========="
5:
    Escribir "  +---+"
    Escribir "  |   |"
    Escribir "  O   |"
    Escribir " /|\  |"
    Escribir " /    |"
    Escribir "      |"
    Escribir "=========="
6:
    Escribir "  +---+"
    Escribir "  |   |"
```

```
        Escribir "  O   |"
        Escribir " /|\  |"
        Escribir " / \  |"
        Escribir "      |"
        Escribir "========="
    FinSegun
FinSubProceso

// ==================== ALGORITMO PRINCIPAL ====================

Algoritmo TallerMecanico
    Definir MAX Como Entero
    MAX <- 50

    // Arreglos paralelos para Repuestos
    Definir rep_codigo, rep_nombre Como Caracter
    Definir rep_precio Como Real
    Definir rep_cantidad Como Entero
    Dimension rep_codigo[50], rep_nombre[50], rep_precio[50], rep_cantidad[50]
    Definir numRepuestos Como Entero
    numRepuestos <- 0

    // Arreglos paralelos para Ordenes
    Definir ord_placa, ord_cliente, ord_cedula, ord_tipo, ord_estado Como Caracter
    Definir ord_horas, ord_costo_hora Como Real
    Definir ord_num_reps, ord_rep_cants Como Entero
```

```
Definir ord_rep_nombres Como Caracter

Dimension ord_placa[50], ord_cliente[50], ord_cedula[50], ord_tipo[50], ord_estado[50]

Dimension ord_horas[50], ord_costo_hora[50], ord_num_reps[50]

Dimension ord_rep_nombres[50, 5], ord_rep_cants[50, 5]

Definir numOrdenes Como Entero

numOrdenes <- 0


// Variables para Ahorcado

Definir pal_facil, pal_dificil, hist_ganador, hist_palabra Como Caracter

Dimension pal_facil[5], pal_dificil[5], hist_ganador[20], hist_palabra[20]

Definir hist_partidas Como Entero

hist_partidas <- 0


pal_facil[1] <- "ORDEN"; pal_facil[2] <- "PLACA"; pal_facil[3] <- "MOTOR"

pal_facil[4] <- "FRENO"; pal_facil[5] <- "LLANTA"

pal_dificil[1] <- "CARBURADOR"; pal_dificil[2] <- "TRANSMISION"; pal_dificil[3] <- "AMORTIGUADOR"

pal_dificil[4] <- "DIFERENCIAL"; pal_dificil[5] <- "ALTERNADOR"


// Variables de Menu

Definir opcion, i, j, k, cantRep Como Entero

Definir precio, horas, costoPorHora, valorTemp Como Real

Definir codigo, nombre, placa, cliente, cedula, tipoServicio Como Caracter

Definir agregar, busqueda, nombreRep Como Caracter

Definir encontrado, repetido Como Logico
```

```
Repetir
    Escribir ""
    Escribir "========================================"
    Escribir "   SISTEMA DE GESTION - TALLER MECANICO"
    Escribir "========================================"
    Escribir "  1. Registrar repuesto"
    Escribir "  2. Registrar orden de trabajo"
    Escribir "  3. Buscar orden"
    Escribir "  4. Ordenar ordenes"
    Escribir "  5. Ver historial"
    Escribir "  6. Total facturado (recursivo)"
    Escribir "  7. Modulo Ahorcado"
    Escribir "  0. Salir"
    Escribir "----------------------------------------"
    Escribir "  Opcion: " Sin Saltar
    Leer opcion

    Segun opcion Hacer
      1: // REGISTRAR REPUESTO
         Si numRepuestos >= MAX Entonces
            Escribir "No hay espacio para mas repuestos."
         Sino
            Escribir "===== REGISTRAR REPUESTO ====="
            Escribir "Codigo: " Sin Saltar
            Leer codigo
            repetido <- Falso
```

```
Para i <- 1 Hasta numRepuestos Hacer
    Si rep_codigo[i] = codigo Entonces
        repetido <- Verdadero
    FinSi
FinPara
Si repetido Entonces
    Escribir "Ya existe un repuesto con ese codigo."
Sino
    Escribir "Nombre: " Sin Saltar
    Leer nombre
    Repetir
        Escribir "Precio unitario: " Sin Saltar
        Leer precio
        Si precio <= 0 Entonces
            Escribir "El precio debe ser mayor a 0."
        FinSi
    Hasta Que precio > 0
    Repetir
        Escribir "Cantidad en stock: " Sin Saltar
        Leer cantRep
        Si cantRep < 0 Entonces
            Escribir "La cantidad no puede ser negativa."
        FinSi
    Hasta Que cantRep >= 0

    numRepuestos <- numRepuestos + 1
```

```
            rep_codigo[numRepuestos] <- codigo
            rep_nombre[numRepuestos] <- nombre
            rep_precio[numRepuestos] <- precio
            rep_cantidad[numRepuestos] <- cantRep
            Escribir "Repuesto registrado correctamente."
        FinSi
    FinSi


2: // REGISTRAR ORDEN
    Si numOrdenes >= MAX Entonces
        Escribir "No hay espacio para mas ordenes."
    Sino
        Escribir "===== REGISTRAR ORDEN DE TRABAJO ====="
        Escribir "Placa del vehiculo: " Sin Saltar
        Leer placa
        Escribir "Nombre del cliente: " Sin Saltar
        Leer cliente
        Escribir "Cedula del cliente: " Sin Saltar
        Leer cedula
        Repetir
            Escribir "Tipo de servicio (preventivo/correctivo/express): " Sin Saltar
            Leer tipoServicio
        Hasta Que tipoServicio="preventivo" O tipoServicio="correctivo" O tipoServicio="express"


        Repetir
            Escribir "Horas de mano de obra: " Sin Saltar
```

```
    Leer horas
Hasta Que horas > 0
Repetir
    Escribir "Costo por hora: " Sin Saltar
    Leer costoPorHora
Hasta Que costoPorHora > 0

numOrdenes <- numOrdenes + 1
ord_placa[numOrdenes] <- placa
ord_cliente[numOrdenes] <- cliente
ord_cedula[numOrdenes] <- cedula
ord_tipo[numOrdenes] <- tipoServicio
ord_horas[numOrdenes] <- horas
ord_costo_hora[numOrdenes] <- costoPorHora
ord_estado[numOrdenes] <- "pendiente"
ord_num_reps[numOrdenes] <- 0

Escribir "Desea agregar repuestos a la orden? (s/n): " Sin Saltar
Leer agregar
Mientras agregar = "s" Y ord_num_reps[numOrdenes] < 5 Hacer
    Si numRepuestos = 0 Entonces
        Escribir "No hay repuestos registrados aun."
        agregar <- "n"
    Sino
        Escribir "Repuestos disponibles:"
        Para i <- 1 Hasta numRepuestos Hacer
```

```
            Escribir "  ", i, ". ", rep_nombre[i], " - $", rep_precio[i]
        FinPara
        Escribir "Nombre del repuesto: " Sin Saltar
        Leer nombreRep
        Repetir
            Escribir "Cantidad: " Sin Saltar
            Leer cantRep
        Hasta Que cantRep > 0

        ord_num_reps[numOrdenes] <- ord_num_reps[numOrdenes] + 1
        ord_rep_nombres[numOrdenes, ord_num_reps[numOrdenes]] <- nombreRep
        ord_rep_cants[numOrdenes, ord_num_reps[numOrdenes]] <- cantRep

        Escribir "Agregar otro repuesto? (s/n): " Sin Saltar
        Leer agregar
      FinSi
    FinMientras
    Escribir "Orden registrada correctamente."
  FinSi

3: // BUSCAR ORDEN
  Escribir "===== BUSCAR ORDEN ====="
  Escribir "1. Buscar por placa"
  Escribir "2. Buscar por cedula"
  Escribir "Opcion: " Sin Saltar
  Definir opcBusqueda Como Entero
```

```
        Leer opcBusqueda

        Escribir "Ingrese dato a buscar: " Sin Saltar

        Leer busqueda

        encontrado <- Falso



        Para i <- 1 Hasta numOrdenes Hacer

          Si (opcBusqueda = 1 Y ord_placa[i] = busqueda) O (opcBusqueda = 2 Y ord_cedula[i] =
busqueda) Entonces

            Escribir "--- Orden #", i, " ---"

            Escribir "Placa:          ", ord_placa[i]

            Escribir "Cliente:        ", ord_cliente[i]

            Escribir "Tipo servicio:   ", ord_tipo[i]

            Escribir "Repuestos usados:"

            Para j <- 1 Hasta ord_num_reps[i] Hacer

              Escribir "  - ", ord_rep_nombres[i, j], " x", ord_rep_cants[i, j]

            FinPara

            valorTemp <- CalcularValorOrden(i, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps,
ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

            Escribir "Valor final:     $", valorTemp

            Escribir "Clasificacion:   ", ObtenerClasificacion(valorTemp)

            encontrado <- Verdadero

          FinSi

        FinPara

        Si No encontrado Entonces

          Escribir "No se encontraron ordenes."

        FinSi
```

```
4: // ORDENAR ORDENES (Burbuja por valor final)

Si numOrdenes < 2 Entonces

Escribir "No hay suficientes ordenes para ordenar."

Sino

Definir tempPlaca, tempCliente, tempCedula, tempTipo, tempEstado, tempRepNombre Como Caracter

Definir tempHoras, tempCosto, valorA, valorB Como Real

Definir tempNumReps, tempRepCant Como Entero



Para i <- 1 Hasta numOrdenes - 1 Hacer

Para j <- 1 Hasta numOrdenes - i Hacer

valorA <- CalcularValorOrden(j, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

valorB <- CalcularValorOrden(j+1, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)



Si valorA < valorB Entonces

// Intercambiar variables simples

tempPlaca <- ord_placa[j]; ord_placa[j] <- ord_placa[j+1]; ord_placa[j+1] <- tempPlaca

tempCliente <- ord_cliente[j]; ord_cliente[j] <- ord_cliente[j+1]; ord_cliente[j+1] <- tempCliente

tempCedula <- ord_cedula[j]; ord_cedula[j] <- ord_cedula[j+1]; ord_cedula[j+1] <- tempCedula

tempTipo <- ord_tipo[j]; ord_tipo[j] <- ord_tipo[j+1]; ord_tipo[j+1] <- tempTipo

tempEstado <- ord_estado[j]; ord_estado[j] <- ord_estado[j+1]; ord_estado[j+1] <- tempEstado

tempHoras <- ord_horas[j]; ord_horas[j] <- ord_horas[j+1]; ord_horas[j+1] <- tempHoras

tempCosto <- ord_costo_hora[j]; ord_costo_hora[j] <- ord_costo_hora[j+1]; ord_costo_hora[j+1] <- tempCosto
```

```
                    tempNumReps <- ord_num_reps[j]; ord_num_reps[j] <- ord_num_reps[j+1];
ord_num_reps[j+1] <- tempNumReps


                    // Intercambiar sub-arreglos de repuestos

                    Para k <- 1 Hasta 5 Hacer

                        tempRepNombre <- ord_rep_nombres[j, k]

                        ord_rep_nombres[j, k] <- ord_rep_nombres[j+1, k]

                        ord_rep_nombres[j+1, k] <- tempRepNombre


                        tempRepCant <- ord_rep_cants[j, k]

                        ord_rep_cants[j, k] <- ord_rep_cants[j+1, k]

                        ord_rep_cants[j+1, k] <- tempRepCant

                    FinPara

                  FinSi

                FinPara

              FinPara

              Escribir "Ordenes ordenadas por mayor costo exitosamente."

            FinSi


          5: // VER HISTORIAL

            Si numOrdenes = 0 Entonces

              Escribir "No hay ordenes registradas."

            Sino

              Escribir "===== HISTORIAL DE ORDENES ====="

              Definir hayBaja Como Logico

              hayBaja <- Falso
```

```
            Para i <- 1 Hasta numOrdenes Hacer

              Escribir "--- Orden #", i, " ---"

              Escribir "Placa:   ", ord_placa[i], " | Cliente: ", ord_cliente[i]

              valorTemp <- CalcularValorOrden(i, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps,
ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

              Escribir "Valor:   $", valorTemp

              Escribir "Estado:  ", ord_estado[i]

              Si valorTemp < 100 Entonces

                hayBaja <- Verdadero

              FinSi

            FinPara



            Escribir "--- Ordenes con baja rentabilidad (< $100) ---"

            Si No hayBaja Entonces

              Escribir "  Ninguna."

            Sino

              Para i <- 1 Hasta numOrdenes Hacer

                valorTemp <- CalcularValorOrden(i, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps,
ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

                Si valorTemp < 100 Entonces

                  Escribir "  Placa: ", ord_placa[i], " - Valor: $", valorTemp

                FinSi

              FinPara

            FinSi

          FinSi



        6: // TOTAL RECURSIVO
```

```
        Definir totalFacturado Como Real

        totalFacturado <- SumarTotalOrdenes(1, numOrdenes, ord_horas, ord_costo_hora, ord_tipo, ord_num_reps, ord_rep_nombres, ord_rep_cants, rep_nombre, rep_precio, numRepuestos)

        Escribir "Total facturado: $", totalFacturado



    7: // AHORCADO

        Definir opcAhorcado, nivel, errores, maxErrores Como Entero

        Definir palabra, letrasUsadas, letra, ganador Como Caracter

        Definir gano, completa, revelada, repetidaLetra, aciertoLetra Como Logico



        Repetir

            Escribir "===== AHORCADO - TALLER MECANICO ====="

            Escribir "1. Jugar"

            Escribir "2. Ver historial de partidas"

            Escribir "0. Volver"

            Escribir "Opcion: " Sin Saltar

            Leer opcAhorcado



            Si opcAhorcado = 1 Entonces

                Repetir

                    Escribir "Seleccione nivel (1. Facil / 2. Dificil): " Sin Saltar

                    Leer nivel

                Hasta Que nivel = 1 O nivel = 2



                Si nivel = 1 Entonces

                    palabra <- pal_facil[Aleatorio(1,5)]
```

```
Sino
    palabra <- pal_dificil[Aleatorio(1,5)]
FinSi

letrasUsadas <- ""
errores <- 0
maxErrores <- 6
gano <- Falso

Mientras errores < maxErrores Y No gano Hacer
    MostrarAhorcadoGrafico(errores)
    Escribir "Palabra: " Sin Saltar
    completa <- Verdadero

    Para i <- 1 Hasta Longitud(palabra) Hacer
        revelada <- Falso
        Para j <- 1 Hasta Longitud(letrasUsadas) Hacer
            Si Subcadena(letrasUsadas, j, j) = Subcadena(palabra, i, i) Entonces
                revelada <- Verdadero
            FinSi
        FinPara
        Si revelada Entonces
            Escribir Subcadena(palabra, i, i), " " Sin Saltar
        Sino
            Escribir "_ " Sin Saltar
            completa <- Falso
```

```
    FinSi
FinPara
Escribir ""

Si completa Entonces
    gano <- Verdadero
Sino
    Escribir "Letras usadas: ", letrasUsadas
    Escribir "Intentos restantes: ", (maxErrores - errores)
    Escribir "Ingrese una letra: " Sin Saltar
    Leer letra
    letra <- Mayusculas(letra)

    repetidaLetra <- Falso
    Para j <- 1 Hasta Longitud(letrasUsadas) Hacer
        Si Subcadena(letrasUsadas, j, j) = letra Entonces
            repetidaLetra <- Verdadero
        FinSi
    FinPara

    Si repetidaLetra Entonces
        Escribir "Esa letra ya fue usada."
    Sino
        letrasUsadas <- letrasUsadas + letra
        aciertoLetra <- Falso
        Para i <- 1 Hasta Longitud(palabra) Hacer
```

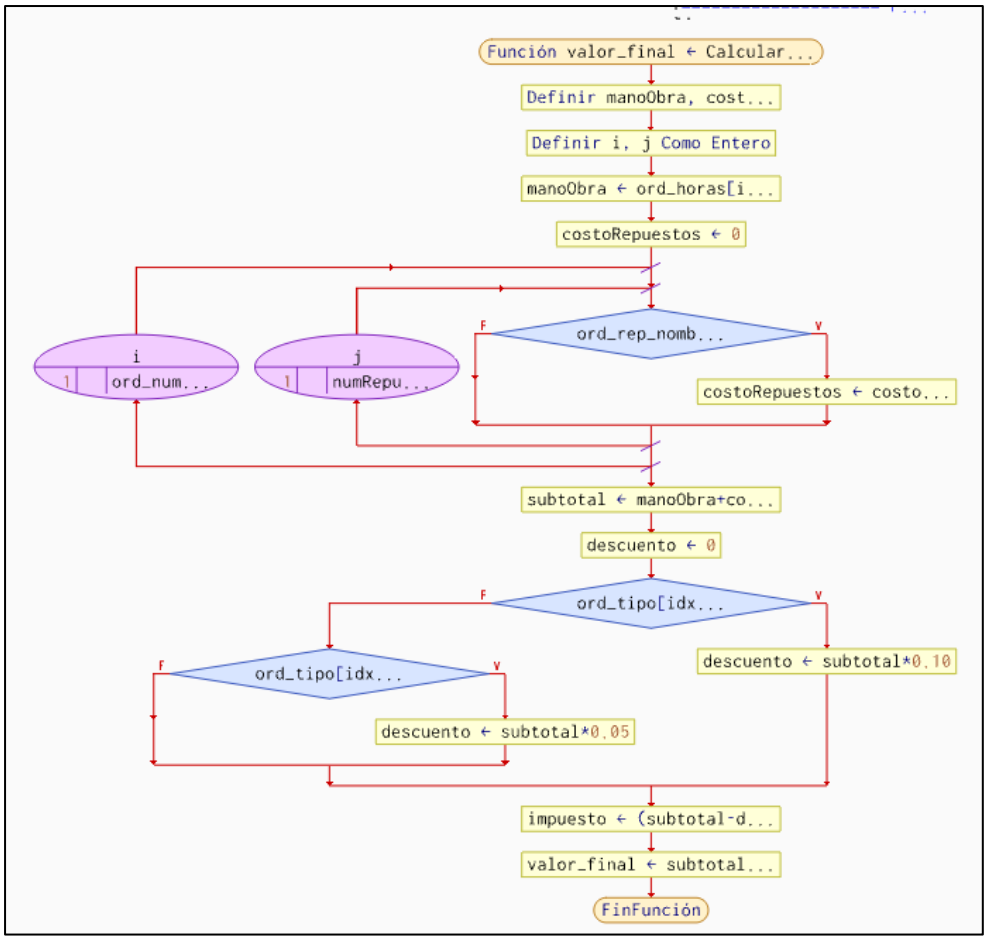
```
                Si Subcadena(palabra, i, i) = letra Entonces
                    aciertoLetra <- Verdadero
                FinSi
                                                                    FinPara


            Si No aciertoLetra Entonces
                errores <- errores + 1
                Escribir "Letra incorrecta."
            Sino
                Escribir "Letra correcta."
            FinSi
        FinSi
    FinSi
FinMientras


MostrarAhorcadoGrafico(errores)
Si gano Entonces
    Escribir "Felicidades! Adivinaste la palabra: ", palabra
    ganador <- "Jugador"
Sino
    Escribir "Perdiste. La palabra era: ", palabra
    ganador <- "CPU"
FinSi
Si hist_partidas < 20 Entonces
    hist_partidas <- hist_partidas + 1
    hist_ganador[hist_partidas] <- ganador
```

```
            hist_palabra[hist_partidas] <- palabra
        FinSi
    Sino
        Si opcAhorcado = 2 Entonces
            Si hist_partidas = 0 Entonces
                Escribir "No hay partidas jugadas."
            Sino
                Definir jugGana, cpuGana Como Entero
                jugGana <- 0; cpuGana <- 0
                Escribir "===== HISTORIAL DE PARTIDAS ====="
                Para i <- 1 Hasta hist_partidas Hacer
                    Escribir i, ". Palabra: ", hist_palabra[i], " - Resultado: ", hist_ganador[i]
                    Si hist_ganador[i] = "Jugador" Entonces
                        jugGana <- jugGana + 1
                    Sino
                        cpuGana <- cpuGana + 1
                    FinSi
                FinPara
                Escribir "Jugador: ", jugGana, " victorias | CPU: ", cpuGana, " victorias"
            FinSi
        FinSi
    FinSi
  Hasta Que opcAhorcado = 0
0:
  Escribir "Cerrando sistema..."
De Otro Modo:
```

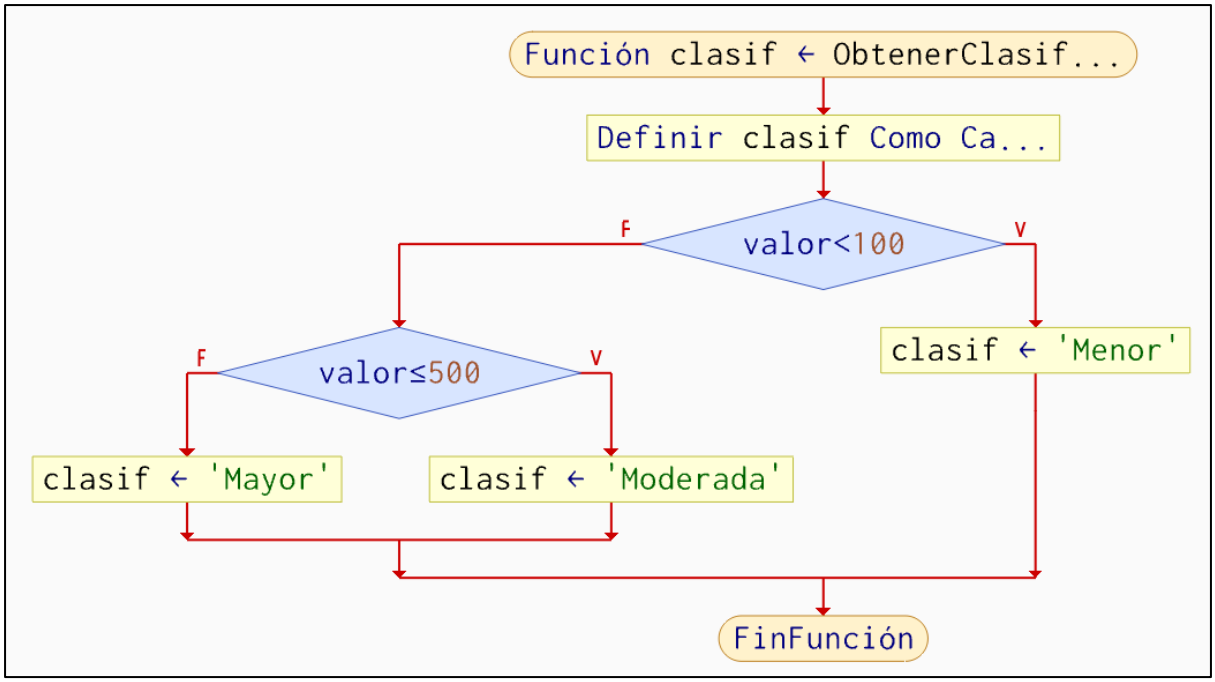
```
            Escribir "Opcion invalida."

        FinSegun

    Hasta Que opcion = 0

FinAlgoritmo
```

Diagramas de Flujo

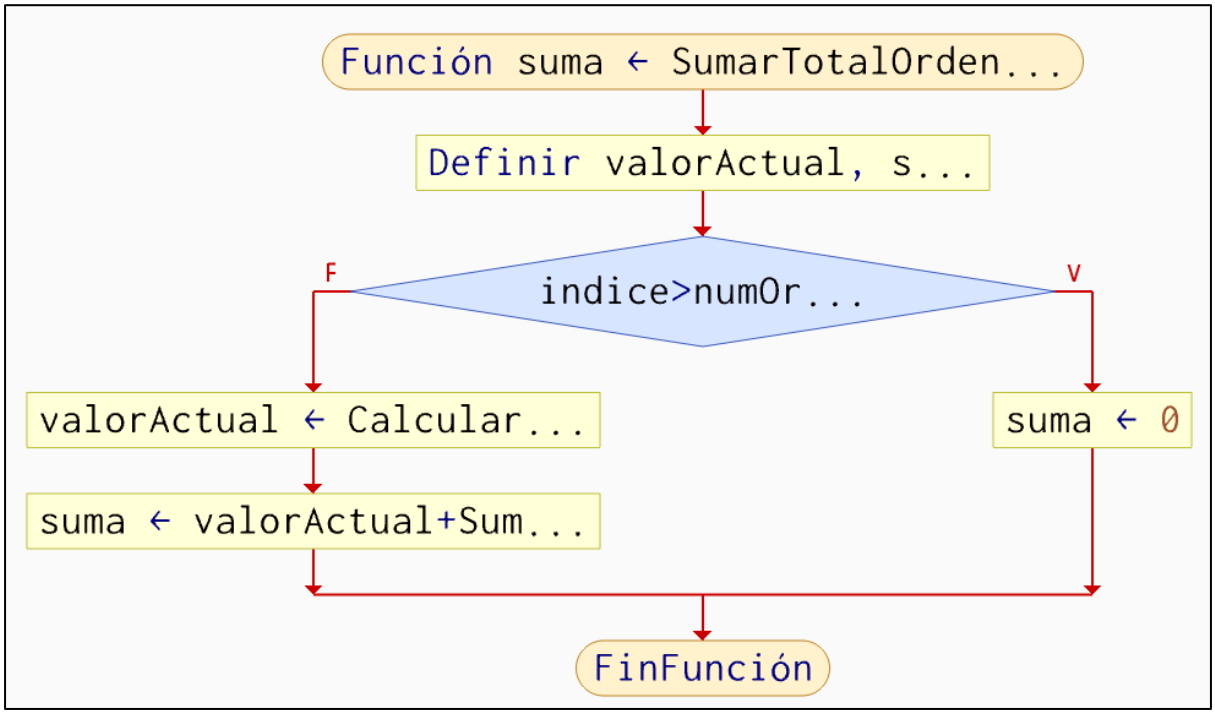
Función valor_final



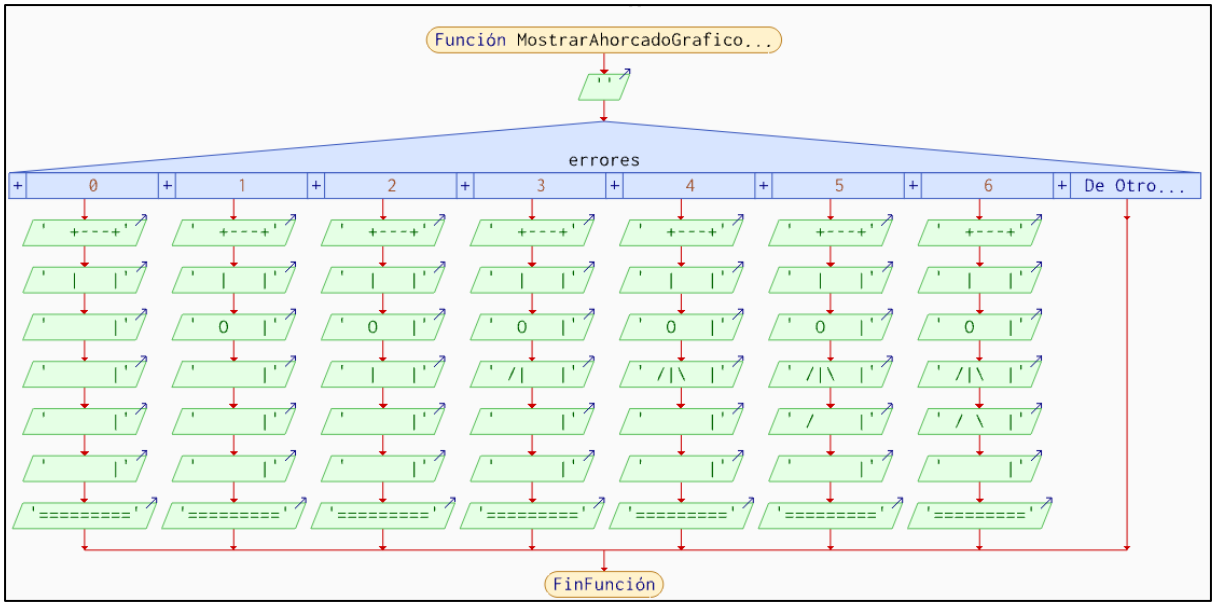
Función clasif



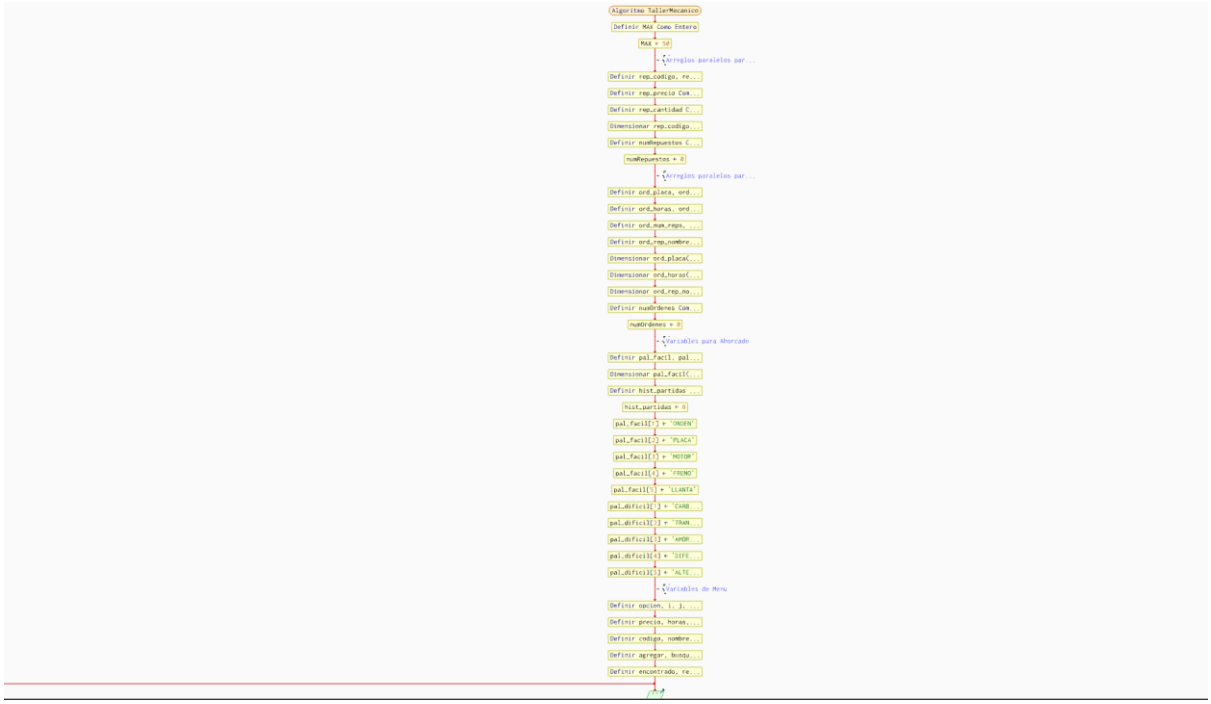
Función suma

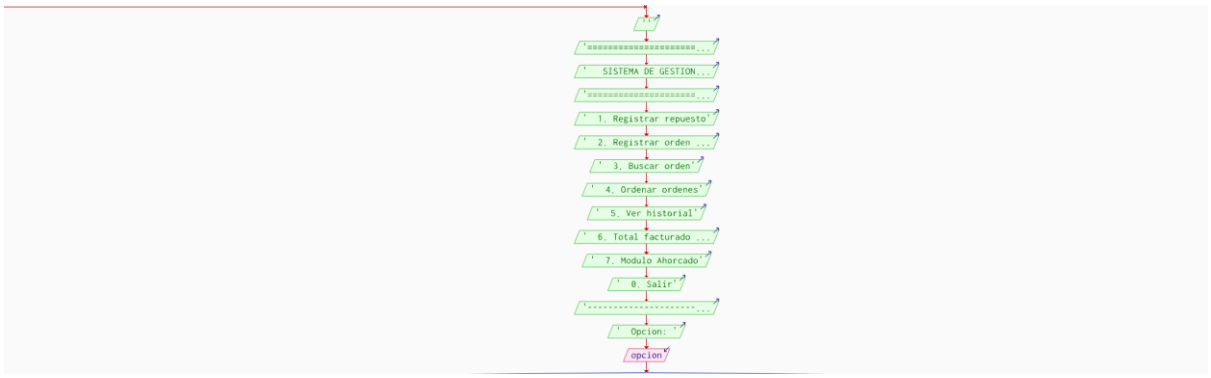


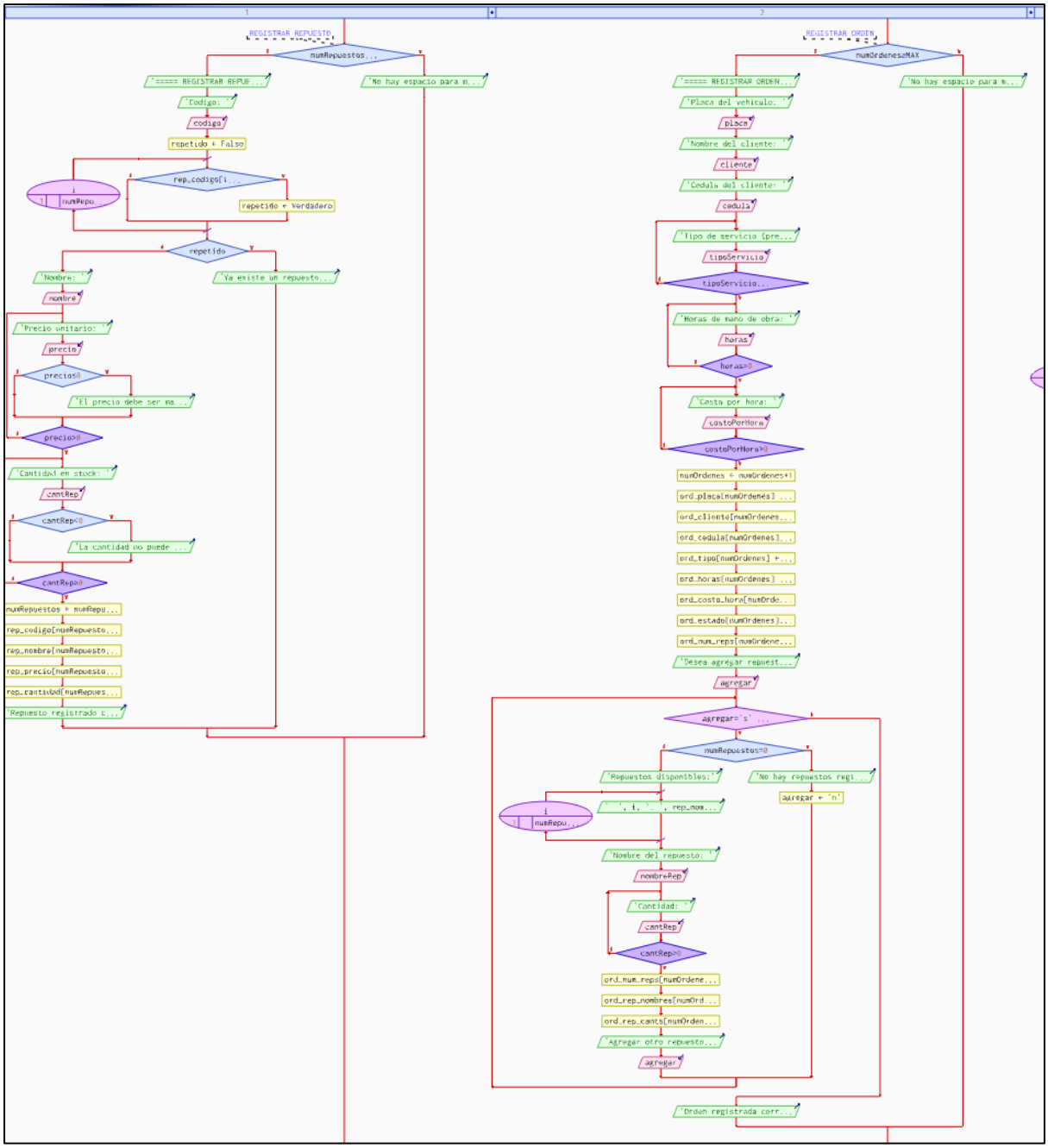
Función MostrarAhorcadoGrafico

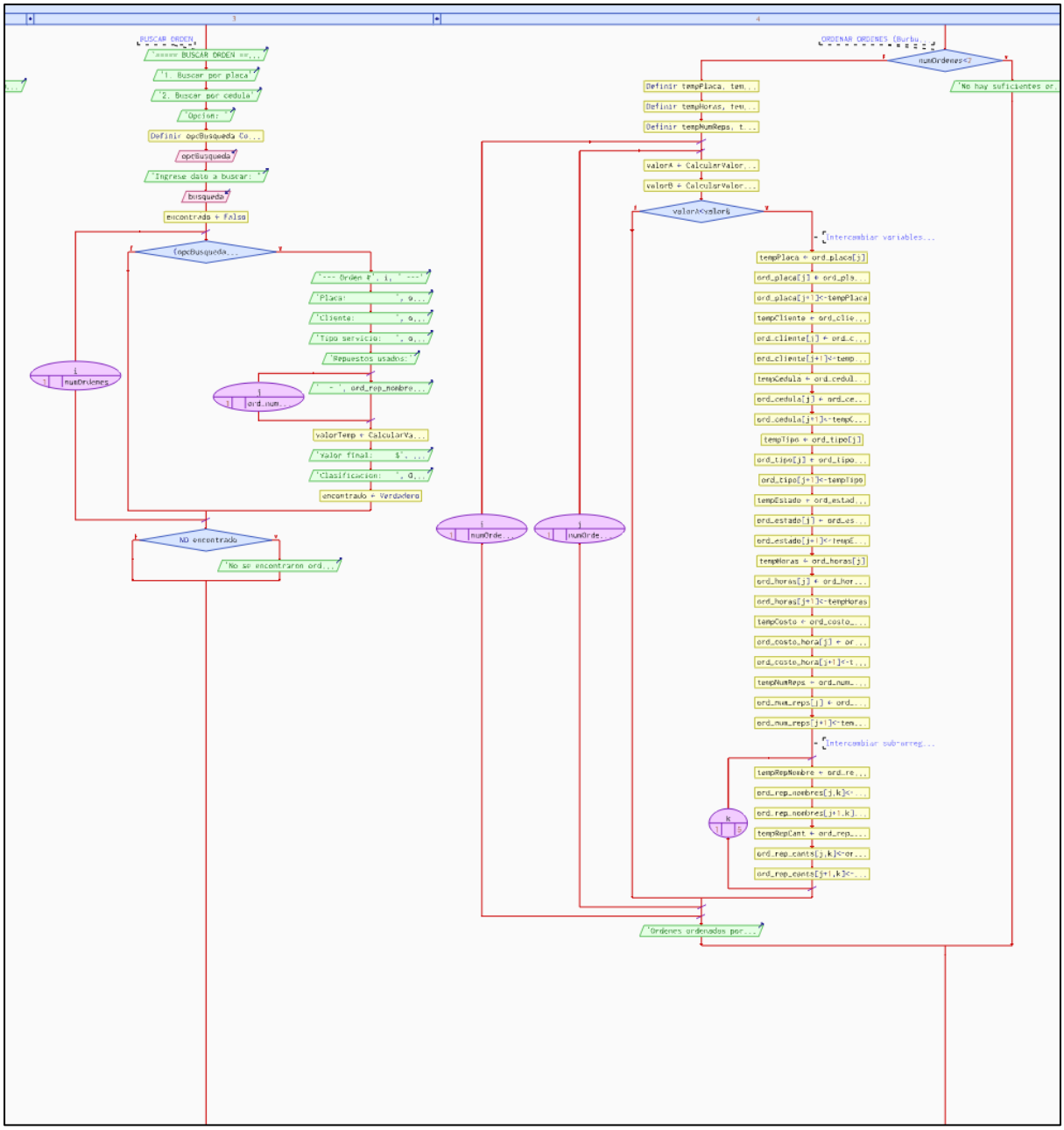


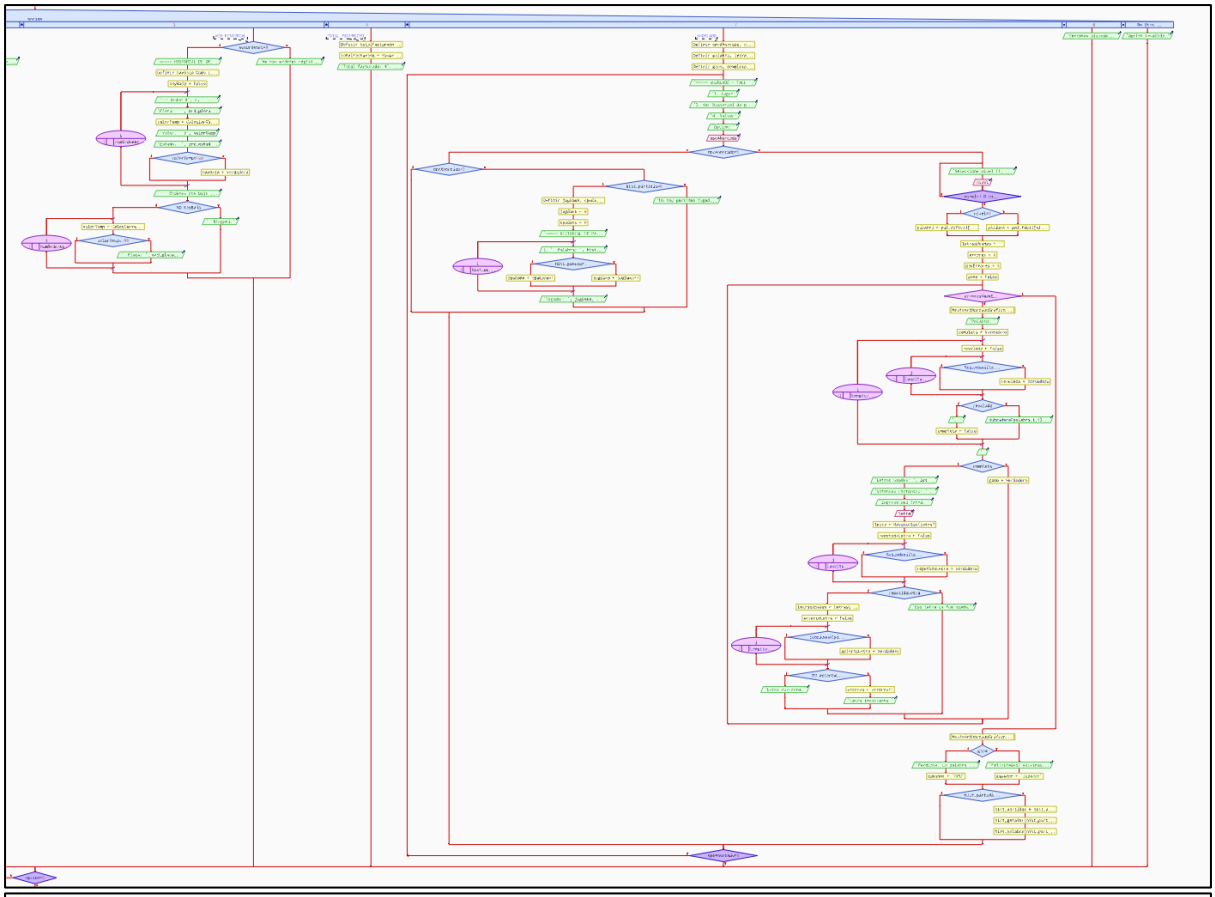
Algoritmo TallerMecanico

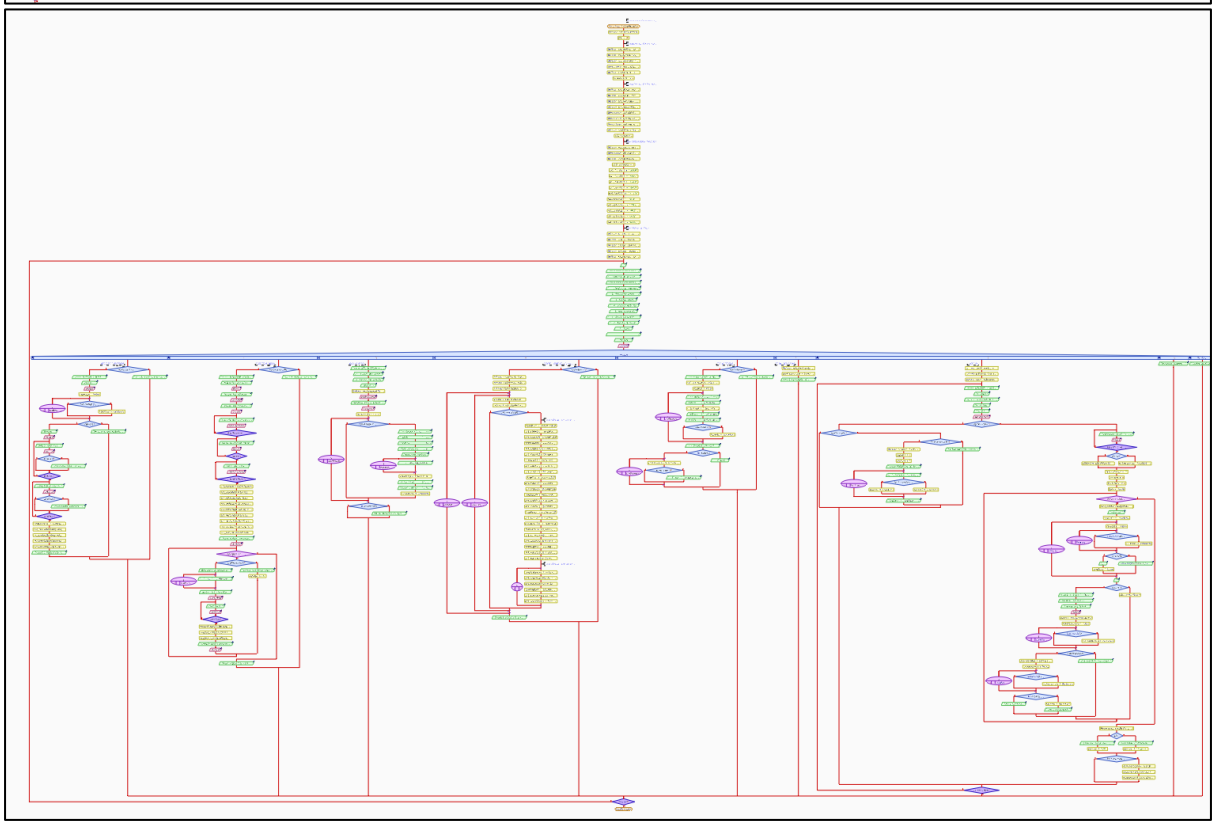





1
2
REGISTRAR REPUESTO
numRepuestos...
'No hay espacio para m...
'===== REGISTRAR REPUE...
'Codigo: '
codigo
repetido ← Falso
rep_codigo[i...
repetido ← Verdadero
numRepu...
repetido
'Ya existe un repuesto...
'Nombre: '
nombre
'Precio unitario: '
precio
precio<0
'El precio debe ser ma...
precio>0
'Cantidad en stock: '
cantRep
cantRep<0
'La cantidad no puede ...
cantRep>0
numRepuestos ← numRepu...
rep_codigo[numRepuesto...
rep_nombre[numRepuesto...
rep_precio[numRepuesto...
rep_cantidad[numRepues...
'Repuesto registrado c...
REGISTRAR ORDEN
numOrdenes<MAX
'No hay espacio para m...
'===== REGISTRAR ORDEN...
'Placa del vehiculo: '
placa
'Nombre del cliente: '
cliente
'Cedula del cliente: '
cedula
'Tipo de servicio (pre...
tipoServicio
tipoServicio...
'Horas de mano de obra: '
horas
horas>0
'Costo por hora: '
costoPorHora
costoPorHora>0
numOrdenes ← numOrdenes+1
ord_placa[numOrdenes] ...
ord_cliente[numOrdenes...
ord_cedula[numOrdenes]...
ord_tipo[numOrdenes] ←...
ord_horas[numOrdenes] ...
ord_costo_hora[numOrde...
ord_estado[numOrdenes]...
ord_num_reps[numOrdene...
'Desea agregar repuest...
agregar
agregar='s' ...
numRepuestos=0
'Repuestos disponibles:'
'No hay repuestos regi...
agregar ← 'n'
' ', i, ' ', rep_nom...
numRepu...
'Nombre del repuesto: '
nombreRep
'Cantidad: '
cantRep
cantRep>0
ord_num_reps[numOrdene...
ord_rep_nombres[numOrd...
ord_rep_cants[numOrden...
'Agregar otro repuesto...
agregar
'Orden registrada corr...


BUSCAR ORDEN
"===== BUSCAR ORDEN ===...
"1. Buscar por placa"
"2. Buscar por cedula"
"Opcion:"
Definir opcBusqueda Co...
opcBusqueda
"Ingrese dato a buscar: "
busqueda
encontrado ← Falso
(opcBusqueda...
"--- Orden #", i, " ---"
"Placa: ", o...
"Cliente: ", o...
"Tipo servicio: ", o...
"Repuestos usados:"
" - ", ord_rep_nombre...
i
1
ord_num...
valorTemp ← CalcularVa...
"Valor final: $", ...
"Clasificacion: ", o...
encontrado ← Verdadero
i
1
numOrdenes
NO encontrado
"No se encontraron ord...
ORDENAR ORDENES (Burbu...
numOrdenes<2
"No hay suficientes or...
Definir tempPlaca, tem...
Definir tempHoras, tem...
Definir tempNumReps, t...
valorA ← CalcularValor...
valorB ← CalcularValor...
valorA<valorB
[Intercambiar variables...
tempPlaca ← ord_placa[j]
ord_placa[j] ← ord_pla...
ord_placa[j+1]<-tempPlaca
tempCliente ← ord_clie...
ord_cliente[j] ← ord_c...
ord_cliente[j+1]<-temp...
tempCedula ← ord_cedul...
ord_cedula[j] ← ord_ce...
ord_cedula[j+1]<-tempC...
tempTipo ← ord_tipo[j]
ord_tipo[j] ← ord_tipo...
ord_tipo[j+1]<-tempTipo
tempEstado ← ord_estad...
ord_estado[j] ← ord_es...
ord_estado[j+1]<-tempE...
tempHoras ← ord_horas[j]
ord_horas[j] ← ord_hor...
ord_horas[j+1]<-tempHoras
tempCosto ← ord_costo_...
ord_costo_hora[j] ← or...
ord_costo_hora[j+1]<-t...
tempNumReps ← ord_num...
ord_num_reps[j] ← ord_...
ord_num_reps[j+1]<-tem...
[Intercambiar sub-arreg...
tempRepNombre ← ord_re...
ord_rep_nombres[j,k]<-...
ord_rep_nombres[j+1,k]...
tempRepCant ← ord_rep_...
ord_rep_cants[j,k]<-or...
ord_rep_cants[j+1,k]<-...
k
1
5
i
1
numOrde...
j
1
numOrde...
"Ordenes ordenadas por...





10. Versionamiento en GitHub

El proyecto fue versionado utilizando GitHub Desktop para Windows. El historial de commits refleja el avance real del desarrollo por fases.

Commit	Mensaje	Integrante	Fase
1	Fase 1: estructura base, clases y menú principal	Chacha Victor	1
2	Fase 2: registros y validaciones de repuestos y ordenes	Chacha Victor	2
3	Fase 3: búsqueda, ordenamiento e historial de ordenes	Landeta Edison	3
4	Fase 4: recursividad y fragmento defectuoso corregido	Landeta Edison	4
5	Fase 5: modulo Ahorcado con vocabulario de taller	Miranda Alan	5
6	Fase 6: README del proyecto	Miranda Alan	6
7	Fase 7: prueba de escritorio caso critico	Chacha Victor	7

Repositorio publicado en: https://github.com/VictorChacha1/Prueba_Grupo-L.git

Readme:

Sistema de Gestión - Taller Mecánico
Grupo L - Algoritmos de Programación / C++
Universidad Técnica de Ambato

Integrantes
Victor Chacha
Imanol Miranda
Edison Landeta

Caso asignado
Sistema de control de órdenes de trabajo, repuestos, horas de mano de obra, estados y costos finales para un taller mecánico.

Metodología seguida
1. Análisis: identificación de clases, atributos y reglas de negocio
2. Diseño: definición de clases con encapsulamiento
3. Construcción incremental: estructura base, registros, reportes, recursividad y Ahorcado
4. Pruebas: casos normales, casos límite y corrección del fragmento defectuoso
5. Versionamiento: commits parciales con GitHub Desktop

Partes implementadas
- Clase Repuesto con encapsulamiento completo
- Clase OrdenTaller con encapsulamiento completo
- Registro y validación de repuestos y órdenes
- Búsqueda por placa y por cédula
- Ordenamiento manual por mayor costo (burbuja)
- Historial de órdenes y detección de baja rentabilidad
- Función recursiva para sumar total facturado
- Corrección del fragmento defectuoso totalOrden()
- Módulo Ahorcado con vocabulario de taller mecánico

Fragmento defectuoso corregido
Función totalOrden() tenía 3 errores:
1. Falta punto y coma en la línea del impuesto
2. Retornaba manoObra en lugar de total
3. No validaba que el descuento no supere el subtotal

Herramientas utilizadas
- C++ con compilador g++ (MSYS2)
- Visual Studio Code
- GitHub Desktop
- GitHub

11. Lista de verificación

Verificación	Cumple
Caso principal funcional	SI
Modulo Ahorcado funcional	SI
Menú integrado	SI
Uso de clases y encapsulamiento	SI
Validaciones correctas	SI
Búsqueda y ordenamiento manual	SI
Recursividad útil	SI
Fragmento defectuoso corregido	SI
Prueba de escritorio entregada	SI
Repositorio visible en GitHub	SI
Commits revisados en GitHub Desktop	SI
README presente	SI

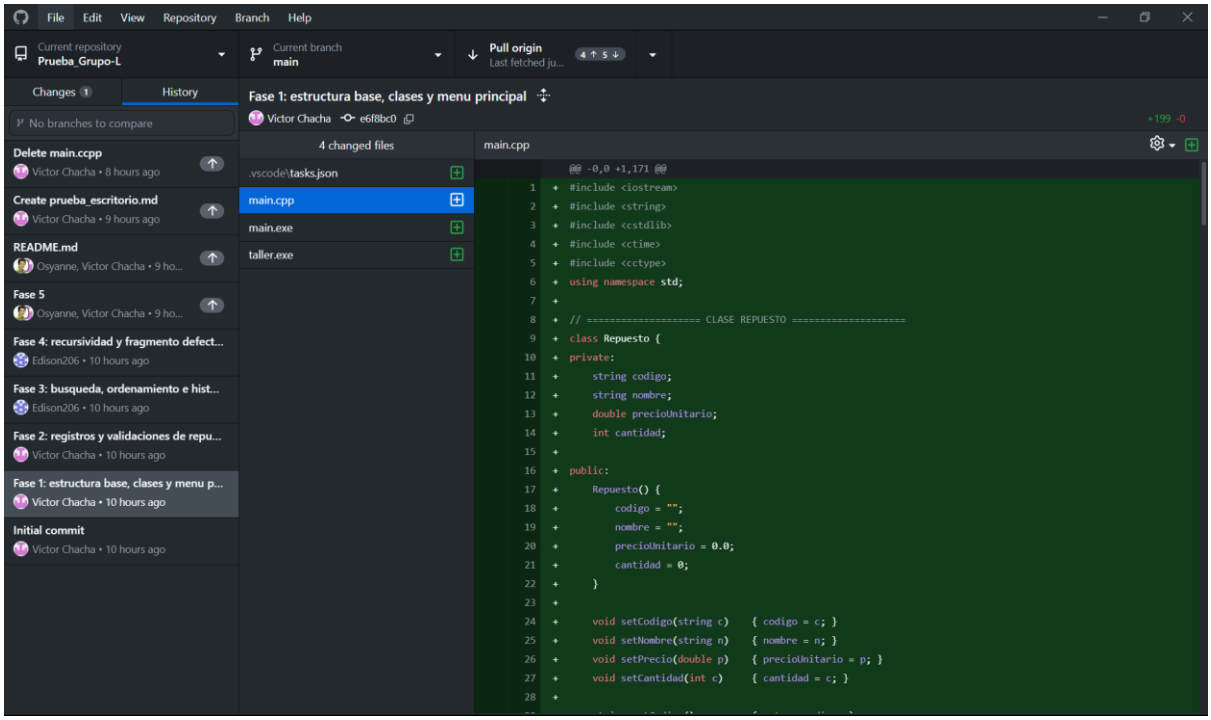
12. Conclusiones

- Se desarrollo exitosamente el sistema de gestión del taller mecánico cumpliendo todos los requisitos técnicos del examen.
- Las clases OrdenTaller y Repuesto fueron implementadas con encapsulamiento completo, constructores, getters y setters.
- El fragmento defectuoso totalOrden() fue corregido identificando y justificando los 3 errores encontrados.
- La función recursiva sumarTotalOrdenes() demuestra una aplicación real de recursividad en el sistema.
- El módulo Ahorcado fue integrado al sistema con vocabulario propio del taller mecanico.
- El versionamiento con GitHub Desktop refleja el proceso real de desarrollo por fases con commits significativos de los tres integrantes.

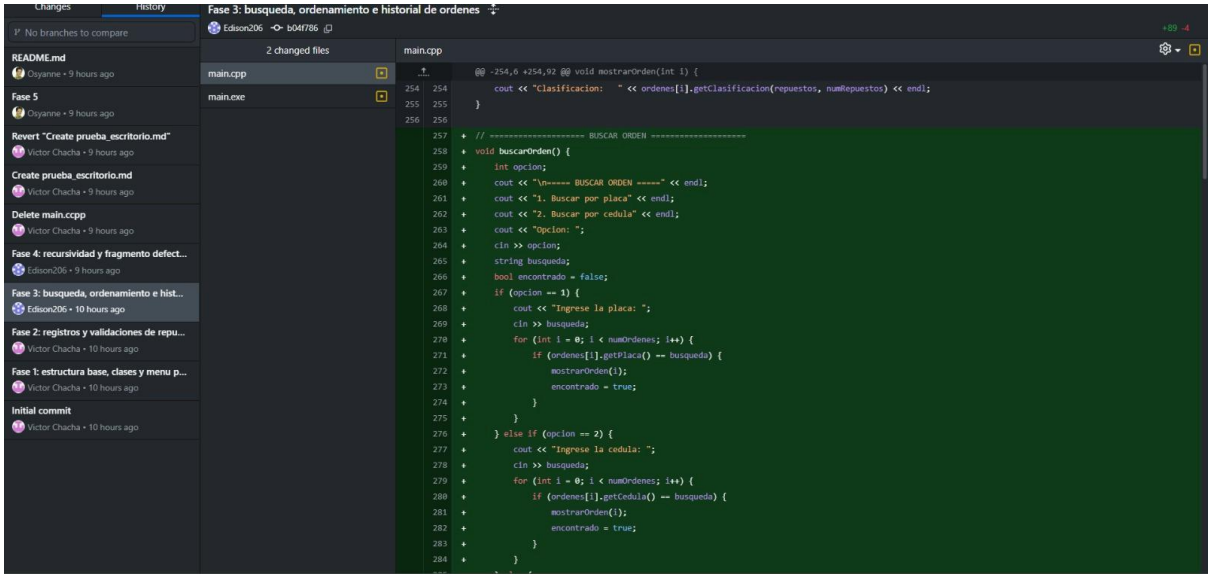
13. Anexos

Desarrollo de actividades mediante el GitHub Desktop

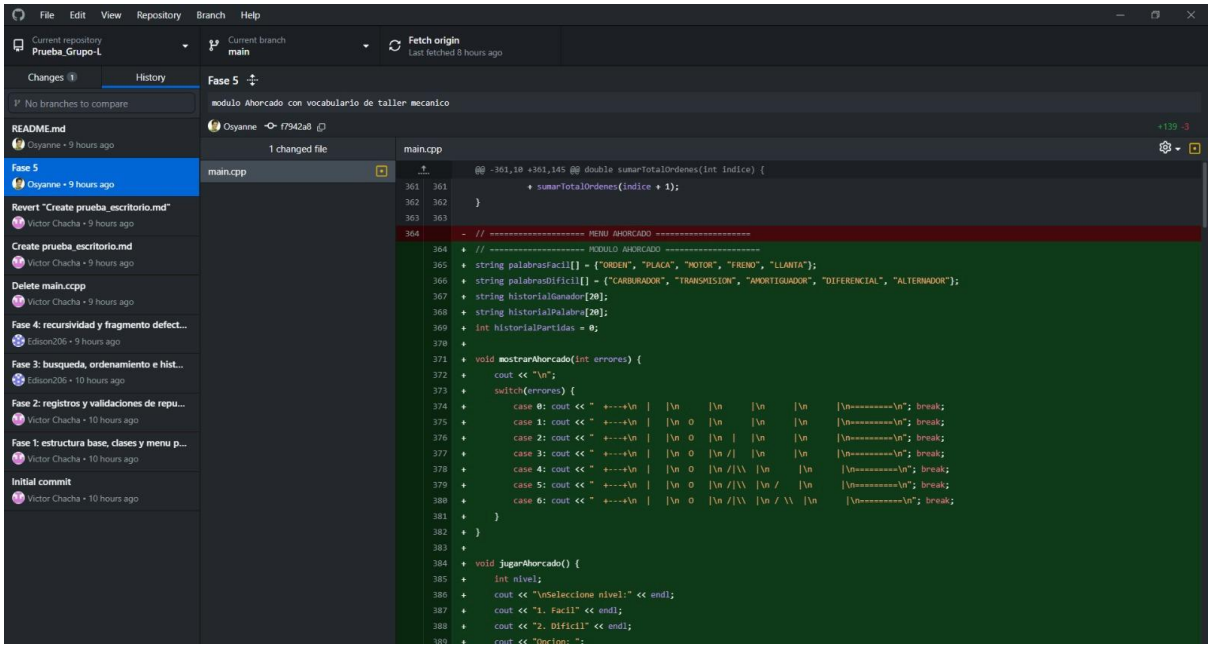
Victor



Edison


Changes
History
No branches to compare
README.md
Osyanne • 9 hours ago
Fase 5
Osyanne • 9 hours ago
Revert "Create prueba_escritorio.md"
Victor Chacha • 9 hours ago
Create prueba_escritorio.md
Victor Chacha • 9 hours ago
Delete main.ccpp
Victor Chacha • 9 hours ago
Fase 4: recursividad y fragmento defect...
Edison206 • 9 hours ago
Fase 3: busqueda, ordenamiento e hist...
Edison206 • 10 hours ago
Fase 2: registros y validaciones de repu...
Victor Chacha • 10 hours ago
Fase 1: estructura base, clases y menu p...
Victor Chacha • 10 hours ago
Initial commit
Victor Chacha • 10 hours ago
Fase 3: busqueda, ordenamiento e historial de ordenes
Edison206 b04f786
+89 -4
2 changed files
main.cpp
main.exe
main.cpp
@@ -254,6 +254,92 @@ void mostrarOrden(int i) {
254 254 cout << "Clasificacion: " << ordenes[i].getClasificacion(repuestos, numRepuestos) << endl;
255 255 }
256 256
+ // ==================== BUSCAR ORDEN ====================
+ void buscarOrden() {
+ int opcion;
+ cout << "\n===== BUSCAR ORDEN =====" << endl;
+ cout << "1. Buscar por placa" << endl;
+ cout << "2. Buscar por cedula" << endl;
+ cout << "Opcion: ";
+ cin >> opcion;
+ string busqueda;
+ bool encontrado = false;
+ if (opcion == 1) {
+ cout << "Ingrese la placa: ";
+ cin >> busqueda;
+ for (int i = 0; i < numOrdenes; i++) {
+ if (ordenes[i].getPlaca() == busqueda) {
+ mostrarOrden(i);
+ encontrado = true;
+ }
+ }
+ } else if (opcion == 2) {
+ cout << "Ingrese la cedula: ";
+ cin >> busqueda;
+ for (int i = 0; i < numOrdenes; i++) {
+ if (ordenes[i].getCedula() == busqueda) {
+ mostrarOrden(i);
+ encontrado = true;
+ }
+ }

Imanol


File Edit View Repository Branch Help
Current repository
Prueba_Grupo-L
Current branch
main
Fetch origin
Last fetched 8 hours ago
Changes 1
History
No branches to compare
Fase 5
modulo Ahorcado con vocabulario de taller mecanico
Osyanne f7942a8
+139 -3
1 changed file
main.cpp
@@ -361,10 +361,145 @@ double sumarTotalOrdenes(int indice) {
361 361 + sumarTotalOrdenes(indice + 1);
362 362 }
363 363
- // ==================== MENU AHORCADO ====================
+ // ==================== MODULO AHORCADO ====================
+ string palabrasFacil[] = {"ORDEN", "PLACA", "MOTOR", "FRENO", "LLANTA"};
+ string palabrasDificil[] = {"CARBURADOR", "TRANSMISION", "AMORTIGUADOR", "DIFERENCIAL", "ALTERNADOR"};
+ string historialGanador[20];
+ string historialPalabra[20];
+ int historialPartidas = 0;
+
+ void mostrarAhorcado(int errores) {
+ cout << "\n";
+ switch(errores) {
+ case 0: cout << " +---+\n | |\n |\n |\n |\n |\n=========\n"; break;
+ case 1: cout << " +---+\n | |\n O |\n |\n |\n |\n=========\n"; break;
+ case 2: cout << " +---+\n | |\n O |\n | |\n |\n |\n=========\n"; break;
+ case 3: cout << " +---+\n | |\n O |\n /| |\n |\n |\n=========\n"; break;
+ case 4: cout << " +---+\n | |\n O |\n /|\\ |\n |\n |\n=========\n"; break;
+ case 5: cout << " +---+\n | |\n O |\n /|\\ |\n / |\n |\n=========\n"; break;
+ case 6: cout << " +---+\n | |\n O |\n /|\\ |\n / \\ |\n |\n=========\n"; break;
+ }
+ }
+
+ void jugarAhorcado() {
+ int nivel;
+ cout << "\nSeleccione nivel:" << endl;
+ cout << "1. Facil" << endl;
+ cout << "2. Dificil" << endl;
+ cout << "Opcion: ";